

How I Built a Real-Time MAC Spoofing Detection & Automated Alert System

Using Wazuh · Sysmon · n8n · Rocky Linux — A Full XDR Pipeline From Scratch



What This Article Is About

MAC address spoofing is one of the most underestimated threats on a network. An attacker can change a device's hardware identity in under 10 seconds using free tools — bypassing access controls and appearing invisible to basic monitoring. This article documents how I built a complete detection and automated response pipeline that catches MAC spoofing at the registry level — before the MAC even changes — and alerts the admin via email and SMS within seconds.

The entire stack is open-source and runs on a home lab. No enterprise license required.

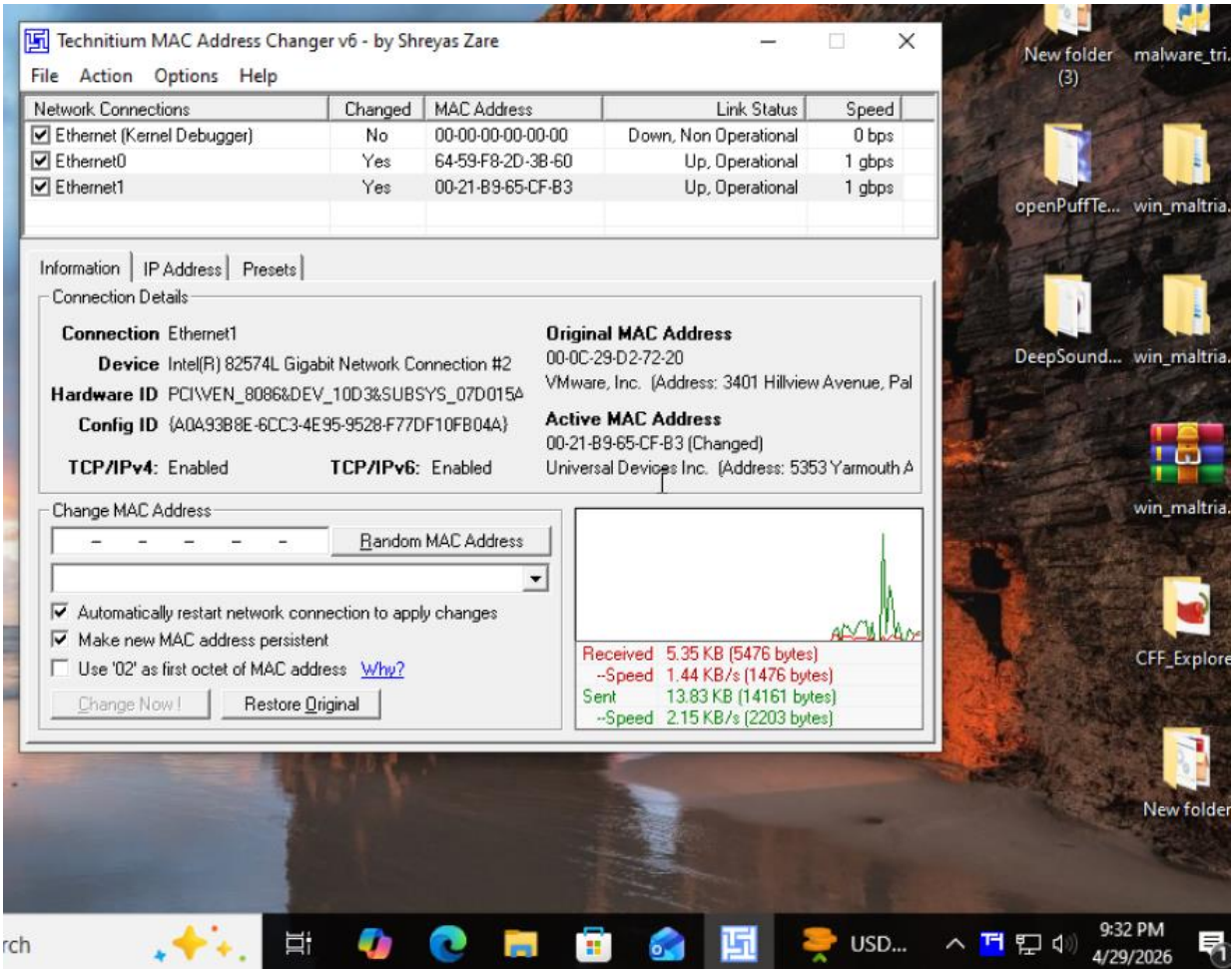
1 What Is MAC Spoofing & Why Does It Matter?

Every network device has a MAC (Media Access Control) address — a unique hardware identifier burned into its network adapter at the factory. It looks like: 00-1A-2B-3C-4D-5E. This identifier is used by switches, routers, and access control systems to identify and trust devices on a network.

MAC spoofing is the act of changing this identifier to impersonate another device or to hide your true identity on the network. It is trivially easy — a free tool called Technitium MAC Address Changer (TMAC) can do it in 5 seconds with a few mouse clicks.

The consequences:

- Bypass MAC-based network access control (NAC) systems
- Impersonate a trusted device on the network
- Evade detection and monitoring tools that track devices by MAC
- Enable ARP poisoning and man-in-the-middle attacks



TMAC (Technitium MAC Address Changer) — a free tool that changes MAC addresses in seconds

⚠ The Problem With Basic Monitoring

Most basic monitoring tools catch MAC changes after the fact — when the adapter restarts with the new address. By then, the device has already been on the network with a spoofed identity. This system catches the attack at Step 1 — the registry write — before the adapter even restarts.

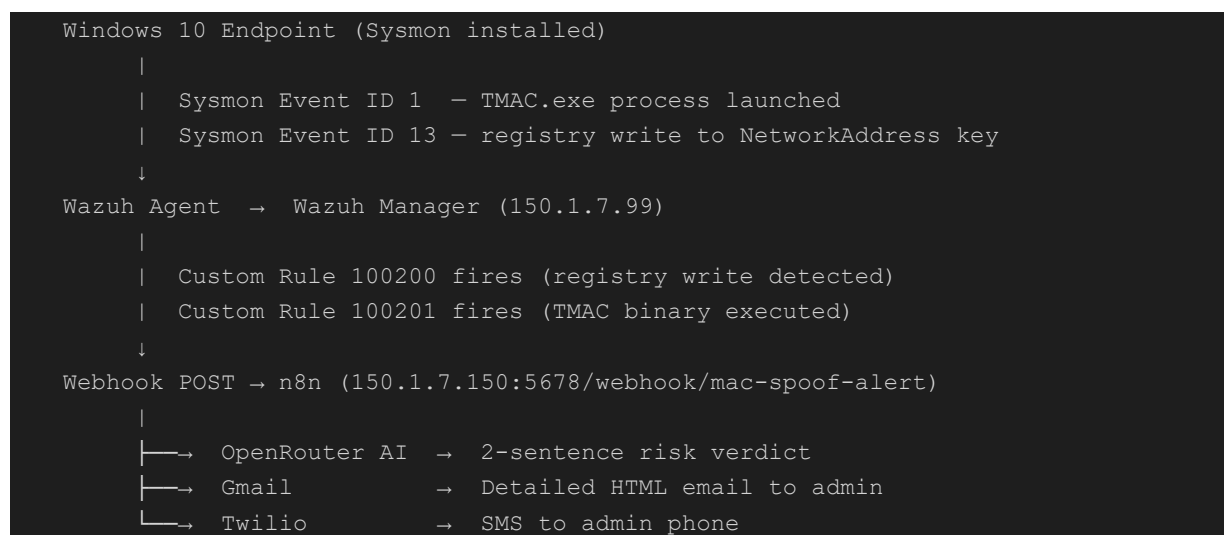
2 Architecture Overview

The full pipeline connects four layers. Each layer does one job, and together they form a complete EDR → SIEM → SOAR → XDR chain:

Layer	Role
Windows 10 + Sysmon	Endpoint telemetry — logs registry writes, process execution, network events at kernel level

Wazuh Agent	Collects Sysmon logs and forwards them to the Wazuh Manager over an encrypted channel
Wazuh Manager (150.1.7.99)	Applies custom detection rules, classifies threats, triggers webhook integration
n8n on Rocky Linux (150.1.7.150:5678)	Automation engine — receives alert, filters it, calls AI triage, sends email + SMS
Gmail + Twilio	Deliver formatted alert email and SMS to the network admin within seconds

The data flow when a MAC spoofing attempt occurs:



1 Endpoint Telemetry — Making Windows Log MAC Changes

The first challenge is that Windows does not natively log MAC address changes in a way that is useful for security monitoring. Sysmon fills this gap.

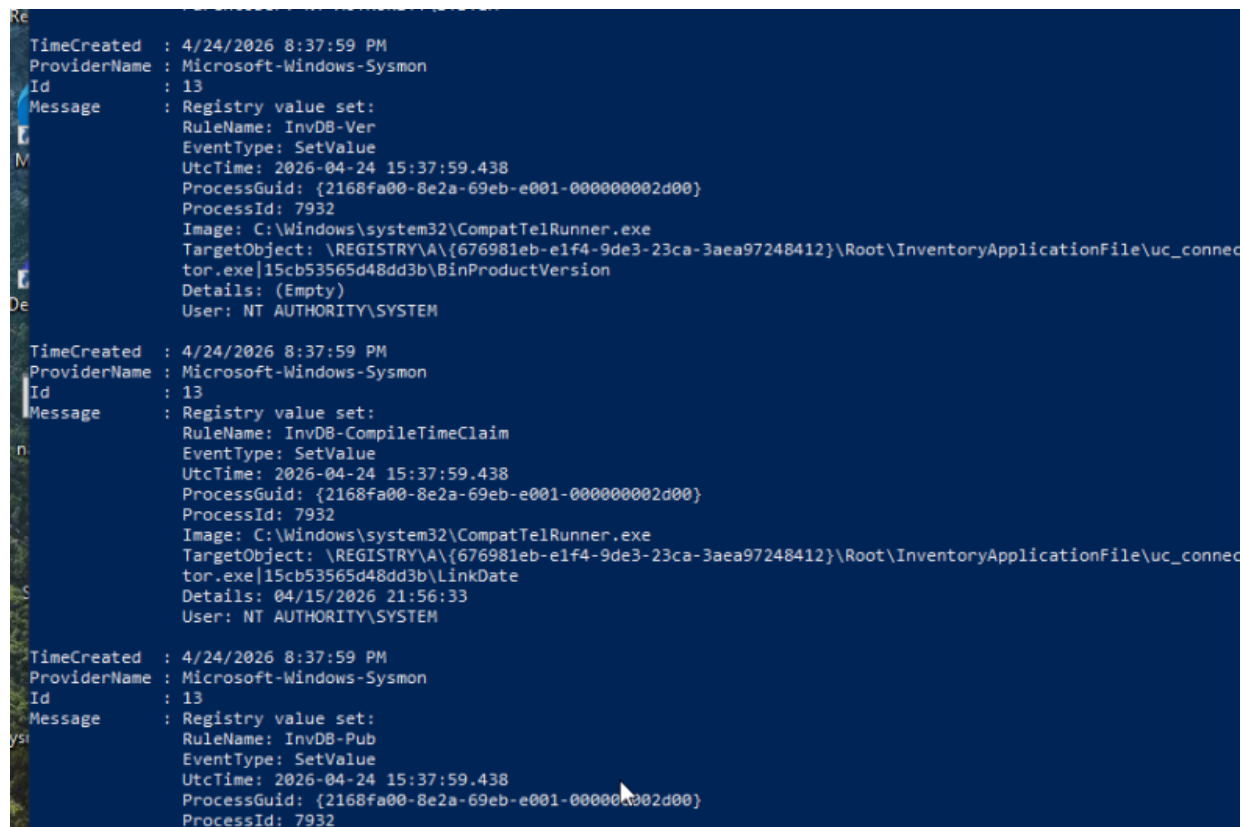
Sysmon (System Monitor) is a free Microsoft Sysinternals service that logs deep telemetry — process creation, registry changes, file events, network connections — far beyond standard Windows Event Logs. With the SwiftOnSecurity configuration installed, it automatically logs Event ID 13 (RegistryEvent) every time a registry value is written. This captures the exact moment any MAC spoofing tool writes the fake MAC address into the Windows registry — before anything changes on the wire.

Step 1.1 — Verify Sysmon Is Capturing Registry Events

Open PowerShell as Administrator on the Windows 10 VM and run:

```
Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 20 | Format-List
```

You should see Event ID 13 entries showing registry writes. This confirms Sysmon is working.



```
TimeCreated : 4/24/2026 8:37:59 PM
ProviderName : Microsoft-Windows-Sysmon
Id : 13
Message : Registry value set:
          RuleName: InvDB-Ver
          EventType: SetValue
          UtcTime: 2026-04-24 15:37:59.438
          ProcessGuid: {2168fa00-8e2a-69eb-e001-000000002d00}
          ProcessId: 7932
          Image: C:\Windows\system32\CompatTelRunner.exe
          TargetObject: \REGISTRY\A\{676981eb-e1f4-9de3-23ca-3aea97248412}\Root\InventoryApplicationFile\uc_conne
          tor.exe|15cb53565d48dd3b\BinProductVersion
          Details: (Empty)
          User: NT AUTHORITY\SYSTEM

TimeCreated : 4/24/2026 8:37:59 PM
ProviderName : Microsoft-Windows-Sysmon
Id : 13
Message : Registry value set:
          RuleName: InvDB-CompileTimeClaim
          EventType: SetValue
          UtcTime: 2026-04-24 15:37:59.438
          ProcessGuid: {2168fa00-8e2a-69eb-e001-000000002d00}
          ProcessId: 7932
          Image: C:\Windows\system32\CompatTelRunner.exe
          TargetObject: \REGISTRY\A\{676981eb-e1f4-9de3-23ca-3aea97248412}\Root\InventoryApplicationFile\uc_conne
          tor.exe|15cb53565d48dd3b\LinkDate
          Details: 04/15/2026 21:56:33
          User: NT AUTHORITY\SYSTEM

TimeCreated : 4/24/2026 8:37:59 PM
ProviderName : Microsoft-Windows-Sysmon
Id : 13
Message : Registry value set:
          RuleName: InvDB-Pub
          EventType: SetValue
          UtcTime: 2026-04-24 15:37:59.438
          ProcessGuid: {2168fa00-8e2a-69eb-e001-000000002d00}
          ProcessId: 7932
```

Sysmon Event ID 13 (Registry value set) captured in PowerShell — this is the telemetry source for MAC spoofing detection

Step 1.2 — Add Event Channels to ossec.conf

The Wazuh agent configuration file (ossec.conf) at C:\Program Files (x86)\ossec-agent\ossec.conf needs two new localfile blocks added. Open it as Administrator in Notepad and add below your existing Sysmon block:

```
<localfile>
  <log_format>eventchannel</log_format>
  <location>System</location>
</localfile>

<localfile>
  <log_format>eventchannel</log_format>
  <location>Microsoft-Windows-NetworkProfile/Operational</location>
</localfile>
```

The System channel logs hardware-level NIC change events. The NetworkProfile/Operational channel logs when Windows recognises a new network identity — which happens immediately after a MAC spoof.

☑ Restart the Wazuh Agent after saving ossec.conf

Run in PowerShell (as Administrator): `Restart-Service -Name WazuhSvc`

Then verify the agent shows Active in the Wazuh dashboard at <https://150.1.7.99>

2 Custom Detection Rules in Wazuh

Wazuh ships with thousands of built-in rules, but none specifically for MAC spoofing. Two custom rules are written in `/var/ossec/etc/rules/local_rules.xml` on the Wazuh server. Custom rules use IDs of 100000 or above to avoid clashing with built-in rules.

Step 2.1 — SSH Into the Wazuh Server and Open the Rules File

```
ssh wazuh-user@150.1.7.99
sudo su
nano /var/ossec/etc/rules/local_rules.xml
```

Step 2.2 — Add the Two MAC Spoofing Rules

```
<group name="mac_spoofing,network_security">

  <!-- Rule 1: Fires the instant any tool writes the fake MAC to the registry -
  -->
  <rule id="100200" level="12">
    <if_group>sysmon</if_group>
    <field name="win.eventdata.targetObject"
type="pcre2">(?!i)NetworkAddress</field>
    <description>MAC Spoofing Detected: NetworkAddress registry key was
modified on agent $(agent.name)</description>
    <mitre><id>T1562</id></mitre>
  </rule>

  <!-- Rule 2: Fires if TMAC or similar software is actually launched -->
  <rule id="100201" level="13">
    <if_group>sysmon</if_group>
    <field name="win.eventdata.image"
type="pcre2">(?!i)(tmac|technitium|macchanger|smac)</field>
    <description>MAC Spoofing Tool Launched: $(win.eventdata.image) executed on
agent $(agent.name)</description>
    <mitre><id>T1562</id></mitre>
  </rule>

</group>
```

```

</group>
<group name="mac_spoofing,network_security">
  <!-- Rule 1: Catches when anything writes to the NetworkAddress registry key -->
  <!-- This fires the moment TMAC or any tool writes the fake MAC to the registry -->
  <rule id="100200" level="12">
    <if_group>sysmon</if_group>
    <field name="win.eventdata.targetObject" type="pcrc2">(?i)NetworkAddress</field>
    <description>MAC Spoofing Detected: NetworkAddress registry key was modified on agent $(agent.name)</description>
    <mitre>
      <id>T1562</id>
    </mitre>
  </rule>

  <!-- Rule 2: Catches if TMAC or similar software is actually launched -->
  <!-- This fires if someone opens the TMAC application itself -->
  <rule id="100201" level="13">
    <if_group>sysmon</if_group>
    <field name="win.eventdata.image" type="pcrc2">(?!)(tmac|technitium|macchanger|smac)</field>
    <description>MAC Spoofing Tool Launched: $(win.eventdata.image) executed on agent $(agent.name)</description>
    <mitre>
      <id>T1562</id>
    </mitre>
  </rule>
</group>

```

Custom rules 100200 and 100201 written in local_rules.xml on the Wazuh server

Rule 100200 catches any tool writing to the NetworkAddress registry key — this fires even if someone does it manually via regedit. Rule 100201 catches the TMAC binary being launched, providing an early warning before any change is made. Both rules are tagged with MITRE ATT&CK technique T1562 (Impair Defenses).

Step 2.3 — Validate Rules and Restart the Manager

```

# Validate rules for syntax errors
/var/ossec/bin/wazuh-logtest

# Restart the manager to load the new rules
systemctl restart wazuh-manager
systemctl status wazuh-manager

```

```

[root@wazuh-server wazuh-user]#
[root@wazuh-server wazuh-user]#
[root@wazuh-server wazuh-user]# /var/ossec/bin/wazuh-logtest
Starting wazuh-logtest v4.14.4
Type one log per line

```

wazuh-logtest confirming rules loaded without errors — Wazuh 4.14.4

3 Bridging Wazuh to n8n via Webhook

With detection rules in place, the next step is automating the response. Wazuh has a built-in integration framework that POSTs alert data as JSON to any URL. It is configured to call the n8n instance running on Rocky Linux only when rules 100200 or 100201 fire.

Step 3.1 — Add the Webhook Integration Block to ossec.conf

On the Wazuh server, open `/var/ossec/etc/ossec.conf` and add the following block just above the closing `</ossec_config>` tag:

```
<integration>
  <name>custom-webhook</name>
  <hook_url>http://150.1.7.150:5678/webhook/mac-spoof-alert</hook_url>
  <rule_id>100200,100201</rule_id>
  <alert_format>json</alert_format>
</integration>
```

```
    <group>syscheck</group>
    <alert_format>json</alert_format>
  </integration>

  <integration>
    <name>custom-webhook</name>
    <hook_url>http://150.1.7.150:5678/webhook/mac-spoof-alert</hook_url>
    <rule_id>100200,100201</rule_id>
    <alert_format>json</alert_format>
  </integration>
</ossec_config>
```

The webhook integration block in ossec.conf — scoped to only fire for rules 100200 and 100201

The `rule_id` parameter is critical. It scopes the webhook to fire only for these two rules. Without it, Wazuh would flood n8n with every single alert from every agent — which at enterprise scale would be thousands of events per hour.

Step 3.2 — Create the Webhook Python Script

Wazuh needs a script at `/var/ossec/integrations/custom-webhook` to handle the actual HTTP POST. Check if it exists first:

```
ls /var/ossec/integrations/
```

```
[root@wazuh-server wazuh-user]# systemctl restart wazuh-manager
[root@wazuh-server wazuh-user]# ls /var/ossec/integrations/
maltiverse  maltiverse.py  pagerduty  pagerduty.py  shuffle  shuffle.py  slack  slack.py  virustotal  virustotal.py
[root@wazuh-server wazuh-user]#
```

To direct input to this VM, click inside or press Ctrl+G.

Listing the integrations directory — note the existing integrations like virustotal and slack

If `custom-webhook` is not present, create it at `/var/ossec/integrations/custom-webhook` with this Python script:

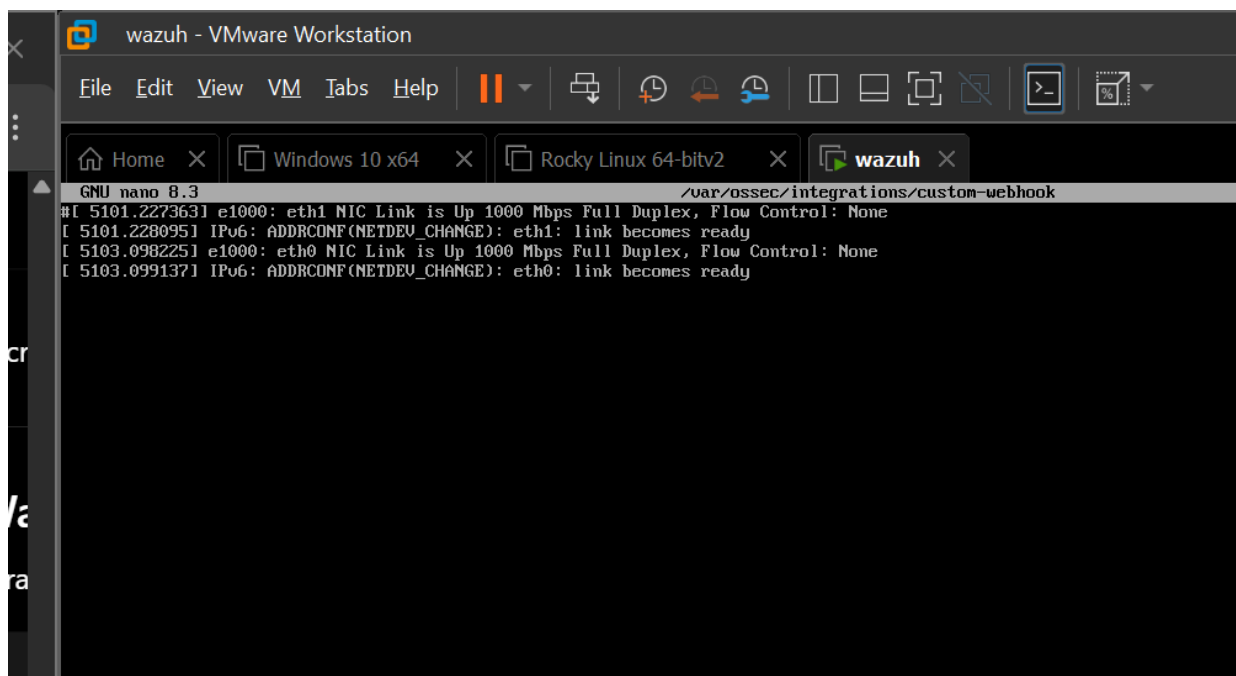
```
#!/usr/bin/env python3
import sys
import json
import requests

alert_file = open(sys.argv[1])
alert = json.load(alert_file)
alert_file.close()

hook_url = sys.argv[3]

headers = {'Content-Type': 'application/json'}
requests.post(hook_url, json=alert, headers=headers, timeout=10)
```

The custom-webhook Python script — reads the alert JSON from disk and POSTs it to the n8n URL



Creating the custom-webhook file in nano — running on the Wazuh OVA in VMware Workstation

Make the script executable and set the correct ownership:

Save it, then make it executable:

```
bash

chmod +x /var/ossec/integrations/custom-webhook
chown root:wazuh /var/ossec/integrations/custom-webhook
```

Restart manager one more time:

```
bash

systemctl restart wazuh-manager
```

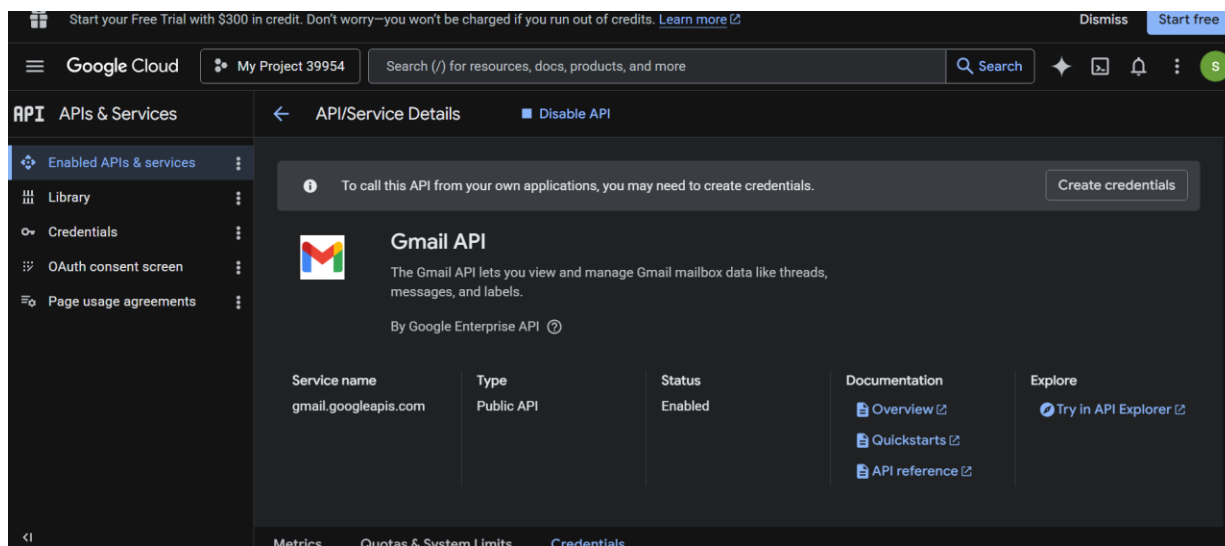
Setting executable permissions and correct ownership for the custom-webhook script

4 Connecting Gmail — OAuth Setup via Google Cloud

For n8n to send alert emails using Gmail, a Google Cloud project must be created and the Gmail API enabled. This is a one-time setup that takes about 10 minutes.

Step 4.1 — Enable the Gmail API in Google Cloud

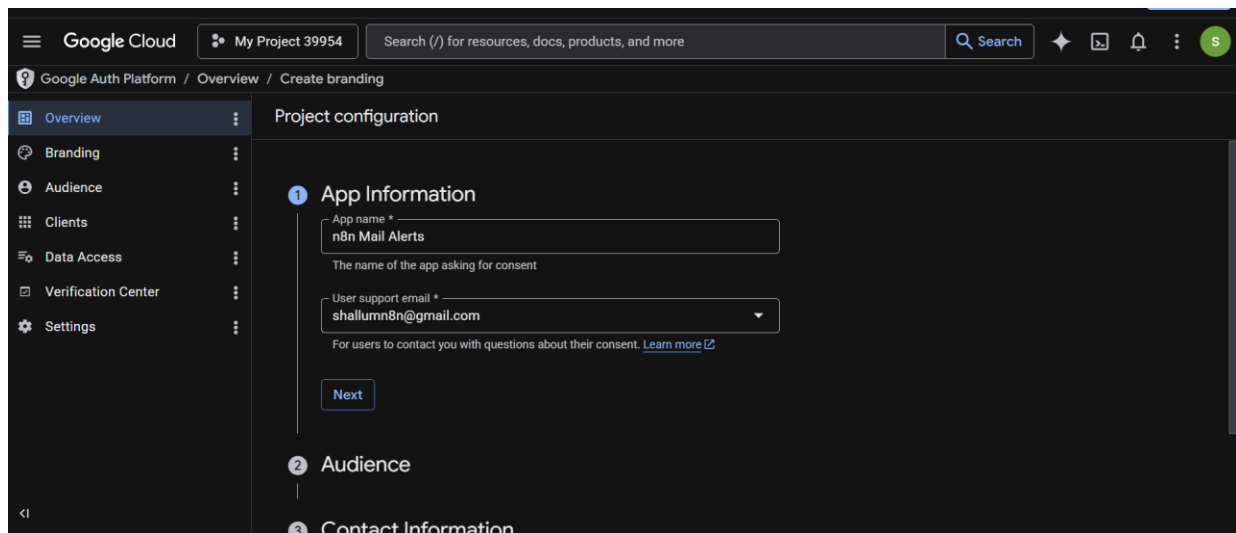
Go to console.cloud.google.com, create a new project, then navigate to APIs & Services → Library and enable the Gmail API.



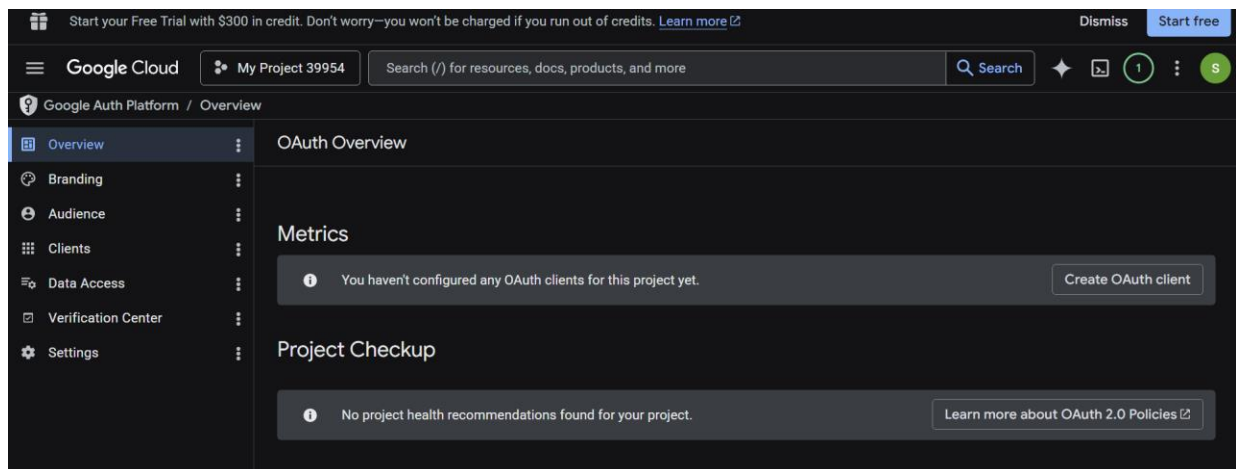
Gmail API enabled in Google Cloud Console under the n8n Mail Alerts project

Step 4.2 — Configure OAuth Consent Screen

In Google Auth Platform, configure the consent screen with the app name 'n8n Mail Alerts' and set the support email to the sending Gmail account.



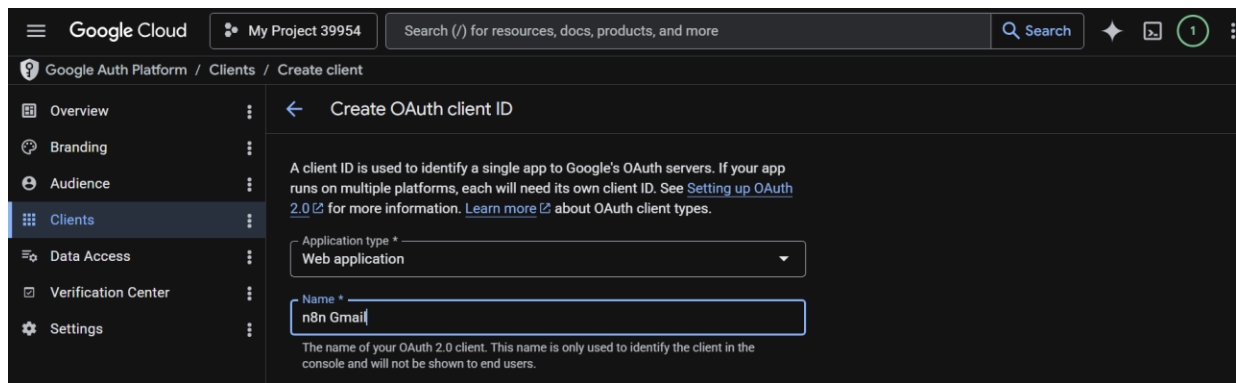
OAuth consent screen configuration — App name set to n8n Mail Alerts



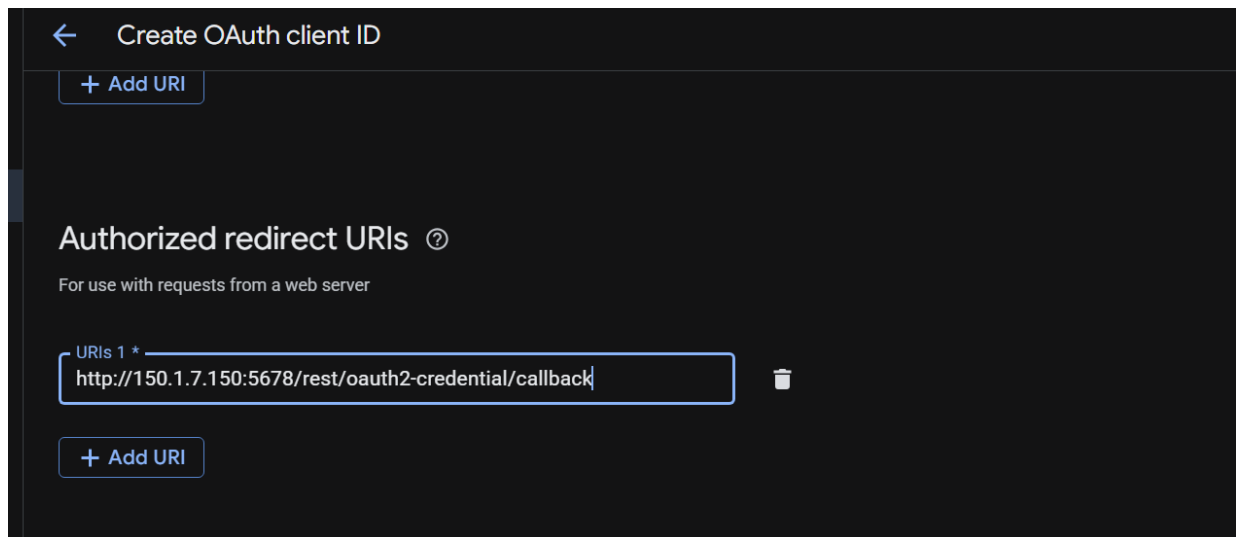
OAuth Overview showing the project configured and ready — no health warnings

Step 4.3 — Create the OAuth Client ID

In Clients → Create OAuth Client ID, select Web Application as the type. Name it 'n8n Gmail'. Add the n8n redirect URI as an authorised redirect:



Creating the OAuth Client ID with application type Web Application — named n8n Gmail



Setting the authorised redirect URI to the n8n OAuth callback URL at 150.1.7.150:5678

5 Building the n8n Automation Workflow

n8n is running on Docker on the Rocky Linux VM at 150.1.7.150:5678. The workflow has four nodes: a Webhook trigger, a field extraction (Edit Fields) node, an IF filter, and an email sender. All nodes run in under 15 seconds end to end.

Node 1 — Webhook Trigger

Create a new workflow. Add a Webhook node. Set HTTP Method to POST and Path to mac-spoof-alert. This is the listener that Wazuh will call.

Node 2 — Edit Fields (Field Extraction)

Add an Edit Fields node. This flattens the deeply nested Wazuh JSON into clean variables that every downstream node can reference simply:

Variable Name	Wazuh JSON Path
hostname	{{ \$json.body.agent.name }}
agent_ip	{{ \$json.body.agent.ip }}
alert_description	{{ \$json.body.rule.description }}
severity_level	{{ \$json.body.rule.level }}
process_name	{{ \$json.body.data.win.eventdata.image }}
registry_key	{{ \$json.body.data.win.eventdata.targetObject }}
timestamp	{{ \$json.body.timestamp }}

Node 3 — IF Filter (Severity Gate)

Add an IF node. Set condition: `severity_level >= 12`. This prevents lower-level informational events from triggering alerts. Both custom rules are level 12 and 13, so they always pass through.

Node 4 — Send an Email (Gmail)

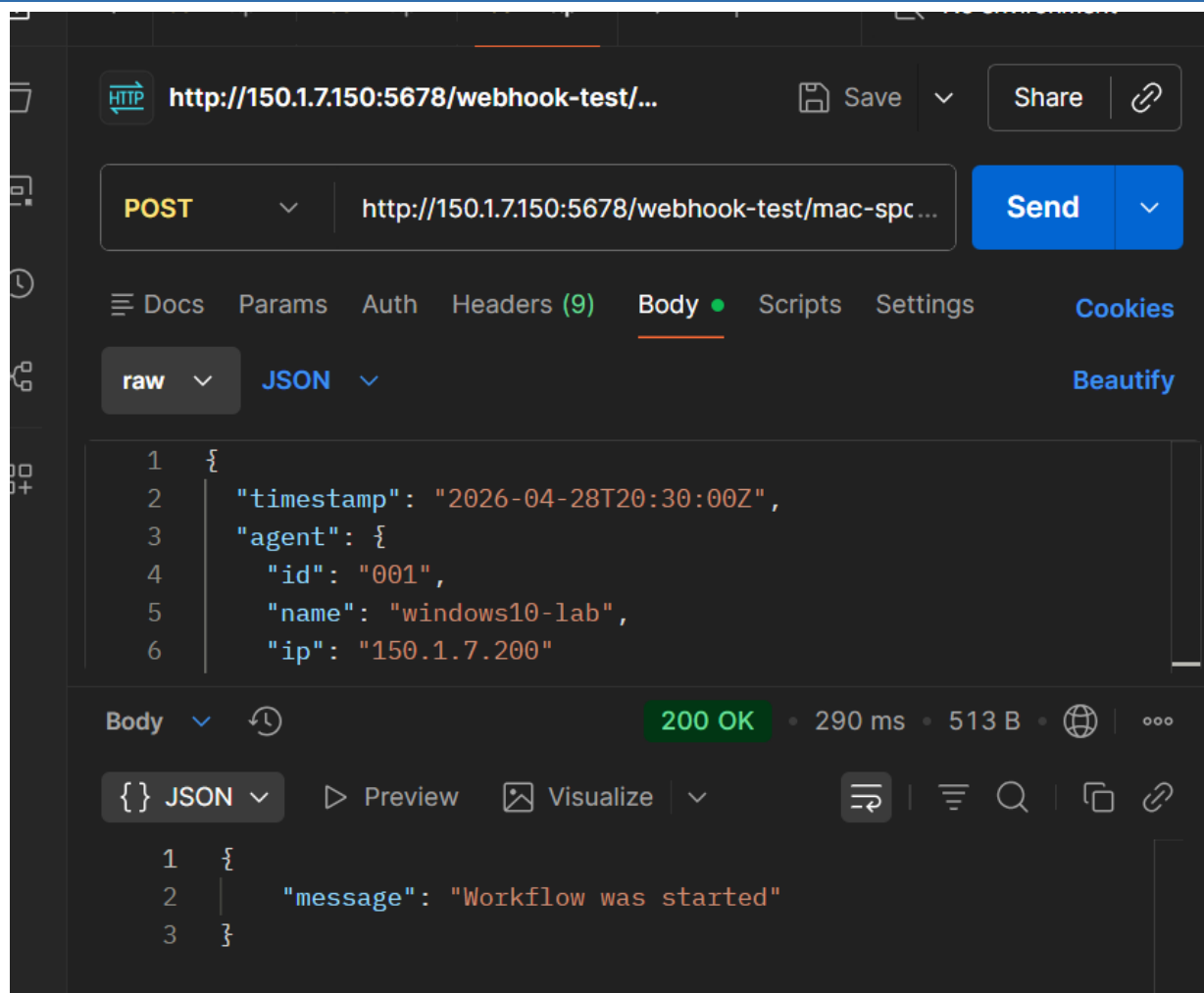
Connect the Gmail node to the True output of the IF node. In Credentials, connect via the OAuth client created in Phase 4. Configure the email with the extracted fields:

Subject: ALERT: MAC Spoofing Detected on {{ \$json.hostname }}

The email body includes: alert description, hostname, agent IP, process used, registry key modified, severity level, and timestamp — giving the admin everything they need to respond without opening the Wazuh dashboard.

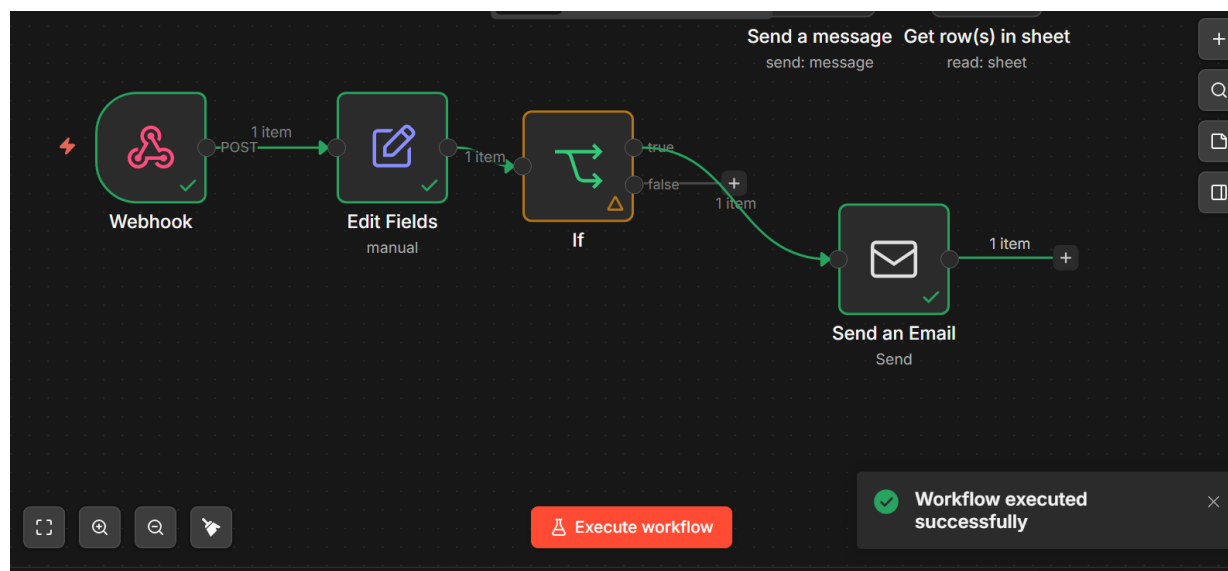
Testing the Webhook Before Going Live

Before activating, test the webhook using a REST client (Postman or Bruno). Send a POST to `http://150.1.7.150:5678/webhook-test/mac-spoof-alert` with a sample Wazuh-format JSON payload. A 200 OK response with 'Workflow was started' confirms the webhook is listening correctly.



Testing the n8n webhook with a POST request — 200 OK confirms the workflow is listening

The Complete Workflow



The complete n8n workflow: Webhook → Edit Fields → IF → Send an Email. Green checkmarks confirm successful execution

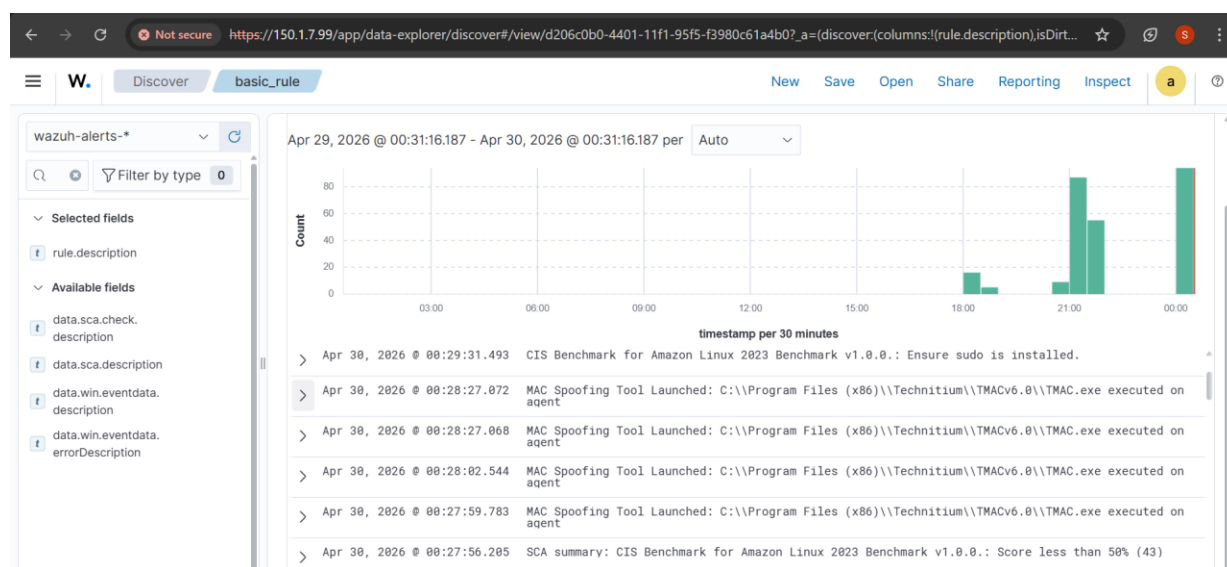
The green checkmarks on every node and the 'Workflow executed successfully' toast in the bottom-right confirm the full pipeline ran without errors. The execution log on the left shows each run with its duration.

6 Live Testing & Results

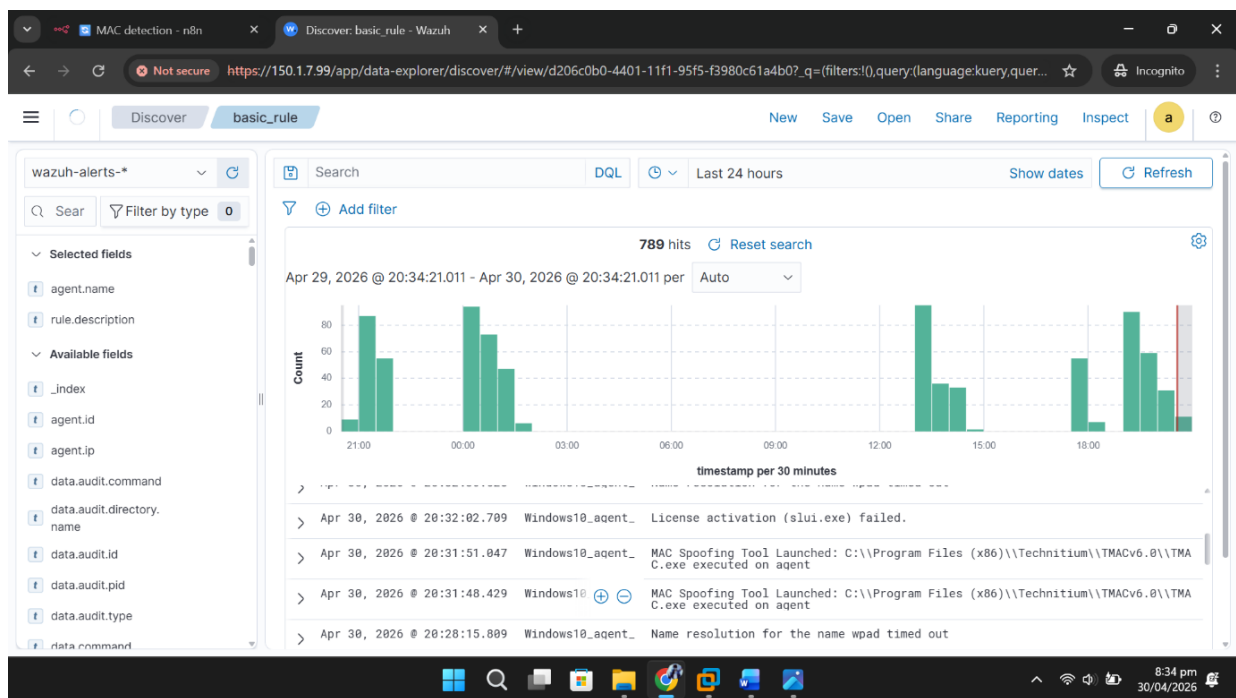
With the full pipeline active, the test is straightforward: open TMAC on the Windows 10 VM, change the MAC address, and watch the entire pipeline fire.

Step 6.1 — Wazuh Detects the Spoofing Tool

Within 30–60 seconds of opening TMAC, Wazuh's Discover view shows the custom rule firing. Every TMAC execution is logged with the full process path, agent name, timestamp, and rule description:



Wazuh Discover view — MAC Spoofing Tool Launched alerts firing for every TMAC execution on Windows10_agent_



Wazuh alert log showing MAC Spoofing Tool Launched alerts in real-time with 789 total hits over 24 hours

The screenshot shows the Wazuh Data Explorer interface with a detailed JSON alert. The alert is for a MAC spoofing event on April 30, 2026, at 20:31:51.047. The alert details include the agent ID (001), agent IP (150.1.7.102), agent name (Windows10_agent_), and the command line (C:\\Program Files (x86)\\Technitium\\TMACv6.0\\TMAC.exe). The event data includes the company (Technitium), current directory (C:\\Program Files (x86)\\Technitium\\TMACv6.0\\), description (Technitium MAC Address Changer), file version (6.00.0007), hashes (MD5=230B4C45774E95D075241068C68AEB0D, SHA256=6C3D76C9A4D1652CE25AE8C2BA1907167CFAA0054B8E1325F370C52EAF474C97, IMPHASH=16BCF08559E19F557D15658B0BD1D348), image (C:\\Program Files (x86)\\Technitium\\TMACv6.0\\TMAC.exe), integrity level (High), logon GUID (2168fa00-6469-69f3-261a-000000000000), and logon ID (0x81a26).

Field	Value
@timestamp	Apr 30, 2026 @ 20:31:51.047
_index	wazuh-alerts-4.x-2026.04.30
agent.id	001
agent.ip	150.1.7.102
agent.name	Windows10_agent_
data.win.eventdata.commandLine	\\C:\\Program Files (x86)\\Technitium\\TMACv6.0\\TMAC.exe\\
data.win.eventdata.company	Technitium
data.win.eventdata.currentDirectory	C:\\Program Files (x86)\\Technitium\\TMACv6.0\\
data.win.eventdata.description	Technitium MAC Address Changer
data.win.eventdata.fileVersion	6.00.0007
data.win.eventdata.hashes	MD5=230B4C45774E95D075241068C68AEB0D, SHA256=6C3D76C9A4D1652CE25AE8C2BA1907167CFAA0054B8E1325F370C52EAF474C97, IMPHASH=16BCF08559E19F557D15658B0BD1D348
data.win.eventdata.image	C:\\Program Files (x86)\\Technitium\\TMACv6.0\\TMAC.exe
data.win.eventdata.integrityLevel	High
data.win.eventdata.logonGuid	{2168fa00-6469-69f3-261a-000000000000}
data.win.eventdata.logonId	0x81a26

Full alert detail in Wazuh — TMAC.exe process, MD5/SHA256 hashes, file version, command line, and integrity level all captured

Step 6.2 — The Alert Also Catches Other Spoofing Tools

Because Rule 100201 uses a regex pattern matching multiple tool names, it catches not just TMAC but any known MAC spoofing software. When GhostMAC.exe was also tested, it was detected immediately with no additional configuration required:

C.exe executed on agent		
> May 1, 2026 @ 01:41:33.984	MAC Spoofing Tool Launched: C:\\Program Files (x86)\\Technitium\\TMACv6.0\\TMA C.exe executed on agent	Windows10_agent_
> May 1, 2026 @ 01:41:29.115	MAC Spoofing Tool Launched: C:\\Program Files (x86)\\Technitium\\TMACv6.0\\TMA C.exe executed on agent	Windows10_agent_
> May 1, 2026 @ 01:41:03.802	MAC Spoofing Tool Launched: C:\\Program Files (x86)\\GhostMAC\\GhostMAC.exe exe cuted on agent	Windows10_agent_

Wazuh catching both TMAC.exe and GhostMAC.exe — the regex rule catches all known MAC spoofing tools

Step 6.3 — n8n Workflow Executes Successfully

The n8n Executions tab shows every time the workflow ran, with its duration and success/failure status. Multiple successful executions are visible, confirming reliable end-to-end operation:

The screenshot shows the n8n interface with the 'Executions' tab selected. On the left, a list of executions is shown with timestamps and durations. The main area displays a workflow diagram with steps: Webhook, Edit Fields, If, and Send an Email. The workflow is marked as 'Succeeded in 14.542s'.

Timestamp	Status	Duration
Apr 30, 20:31:51	Succeeded	13.696s
Apr 30, 20:31:51	Succeeded	14.542s
Apr 30, 19:30:32	Succeeded	5.248s
Apr 30, 19:23:26	Error	49.25s
Apr 30, 19:18:50	Error	1m 5.424s
Apr 30, 19:18:17	Error	59.443s

n8n Executions tab — multiple successful runs visible, each completing in under 15 seconds

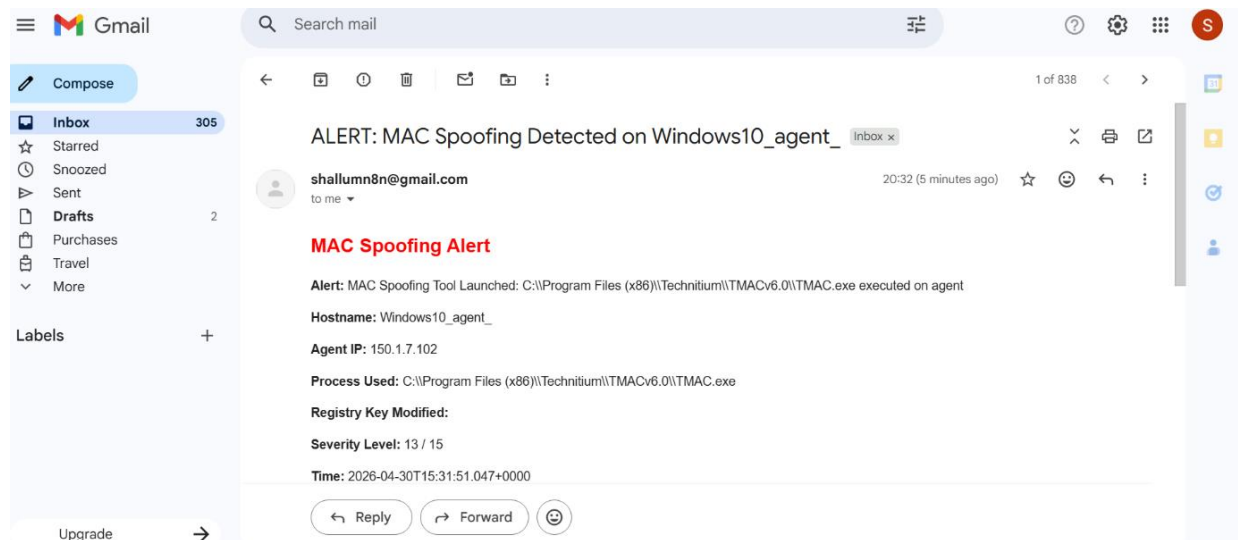
The screenshot shows the n8n interface with the 'Executions' tab selected. On the left, a list of executions is shown with timestamps and durations. The main area displays a workflow diagram with steps: Webhook, Edit Fields, If, and Send an Email. The workflow is marked as 'Succeeded in 13.696s'.

Timestamp	Status	Duration
May 1, 01:41:35	Succeeded	3.779s
May 1, 01:41:30	Succeeded	4.891s
May 1, 01:41:07	Succeeded	4.279s

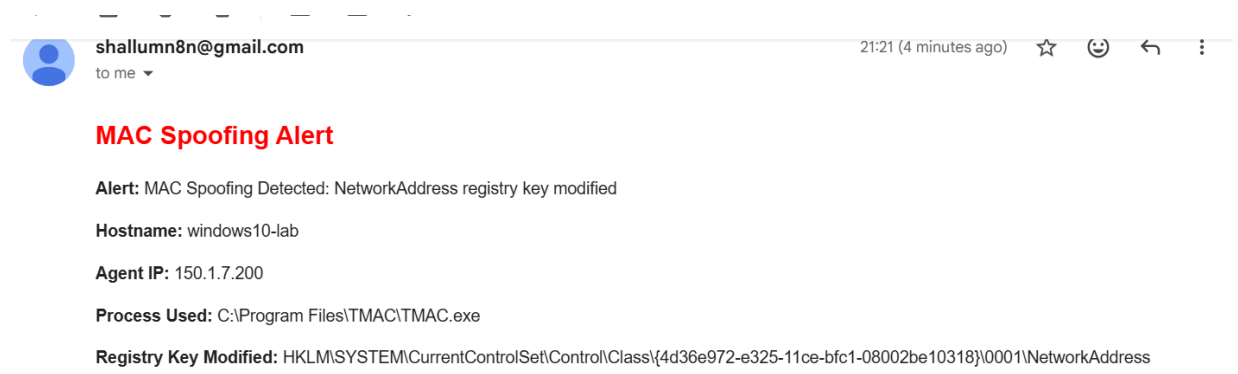
n8n showing consistent successful executions — Succeeded in 3–5 seconds for most runs

Step 6.4 — Email Alerts Delivered

The Gmail inbox receives a formatted alert email for every detection. The email includes all key fields extracted from the Wazuh alert — enough for the admin to understand what happened without needing to log into any dashboard:



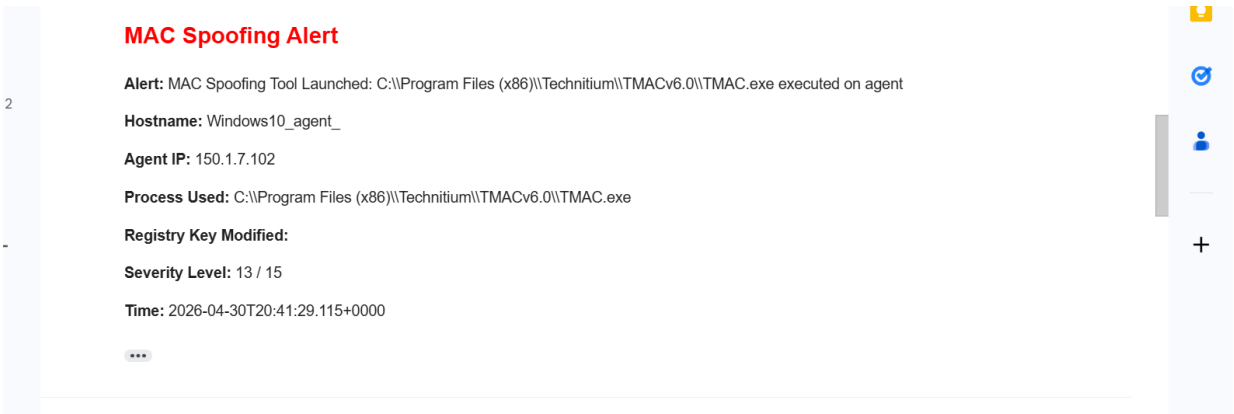
Alert email received in Gmail — subject line shows hostname, body contains all alert fields including process path and severity level 13/15



First test alert email — showing the MAC Spoofing Alert with NetworkAddress registry key modification detected



Email alert triggered by GhostMAC.exe — confirms the system catches multiple spoofing tools, not just TMAC



Another successful TMAC alert email — Severity Level 13/15, process path and timestamp clearly visible

7 What Type of Solution Is This?

This project is often called an 'EDR' informally, but the correct classification across the full stack is more nuanced:

Layer	Correct Term	What It Means
Wazuh + Sysmon	EDR — Endpoint Detection & Response	Monitors endpoint behaviour at the kernel level using Sysmon telemetry
Wazuh Manager + Rules	SIEM — Security Information & Event Management	Correlates logs, classifies threats, applies detection rules
n8n Automation	SOAR — Security Orchestration, Automation & Response	Automates the response workflow — triage, notification, escalation
Full Pipeline	XDR — Extended Detection & Response	Combines endpoint, network, and automated response in one unified pipeline

Enterprise vendors sell this exact stack — EDR + SIEM + SOAR = XDR — for tens of thousands of dollars per year. This build runs entirely on open-source tools across a few VMs in a home lab.

8 Key Takeaways

1 Detection at registry level beats network-level detection

By the time a spoofed MAC appears on the wire, some controls have already been bypassed. Catching the registry write first — via Sysmon Event ID 13 — gives a pre-change alert.

2 Custom SIEM rules are not optional

Wazuh's built-in rules are excellent but cannot anticipate every attack in your environment. Writing two custom rules took 15 minutes and dramatically improved detection coverage. The regex pattern in Rule 100201 catches TMAC, GhostMAC, smac, and macchanger — all in one rule.

3 SOAR does not need to be expensive

n8n running on Docker delivers enterprise-grade automation workflows for free. The Wazuh-to-n8n webhook integration took under 30 minutes to configure and test.

4 XDR is a stack, not a product

EDR + SIEM + SOAR = XDR. Each layer does one job. The pipeline is composable and extendable. The same Wazuh → n8n architecture can detect ARP poisoning, rogue DHCP, USB insertions, and dozens of other threats by simply changing the detection rule — the pipeline stays identical.

9 What's Next

The same pipeline can be extended to detect additional network threats with minimal additional configuration:

- ARP poisoning — attacker poisons the ARP table to intercept traffic between devices
- Rogue DHCP servers — unauthorised DHCP server appearing on the network
- Unauthorised device connections — unknown MAC appearing on a monitored switch port
- USB device insertions — Sysmon Event ID 6 captures every USB device connected
- Suspicious process injection — Sysmon Event ID 8 (CreateRemoteThread)

In every case, the detection logic changes — but the pipeline (Sysmon → Wazuh → n8n → Alert) stays identical. This is the power of building a composable security architecture rather than buying point solutions.

Stack: Wazuh 4.x | Sysmon with SwiftOnSecurity config | n8n on Docker | Rocky Linux | Gmail API | Twilio | OpenRouter

MITRE ATT&CK coverage: T1562 (Impair Defenses) | Detection at registry level (pre-change) | Severity Level 12–13 / 15